Studio Nx

FIX-01 SweetAlert 알림 · 확인 · 입력창 — 함수 설명 합본 (18~21)

2026-10-07

목차

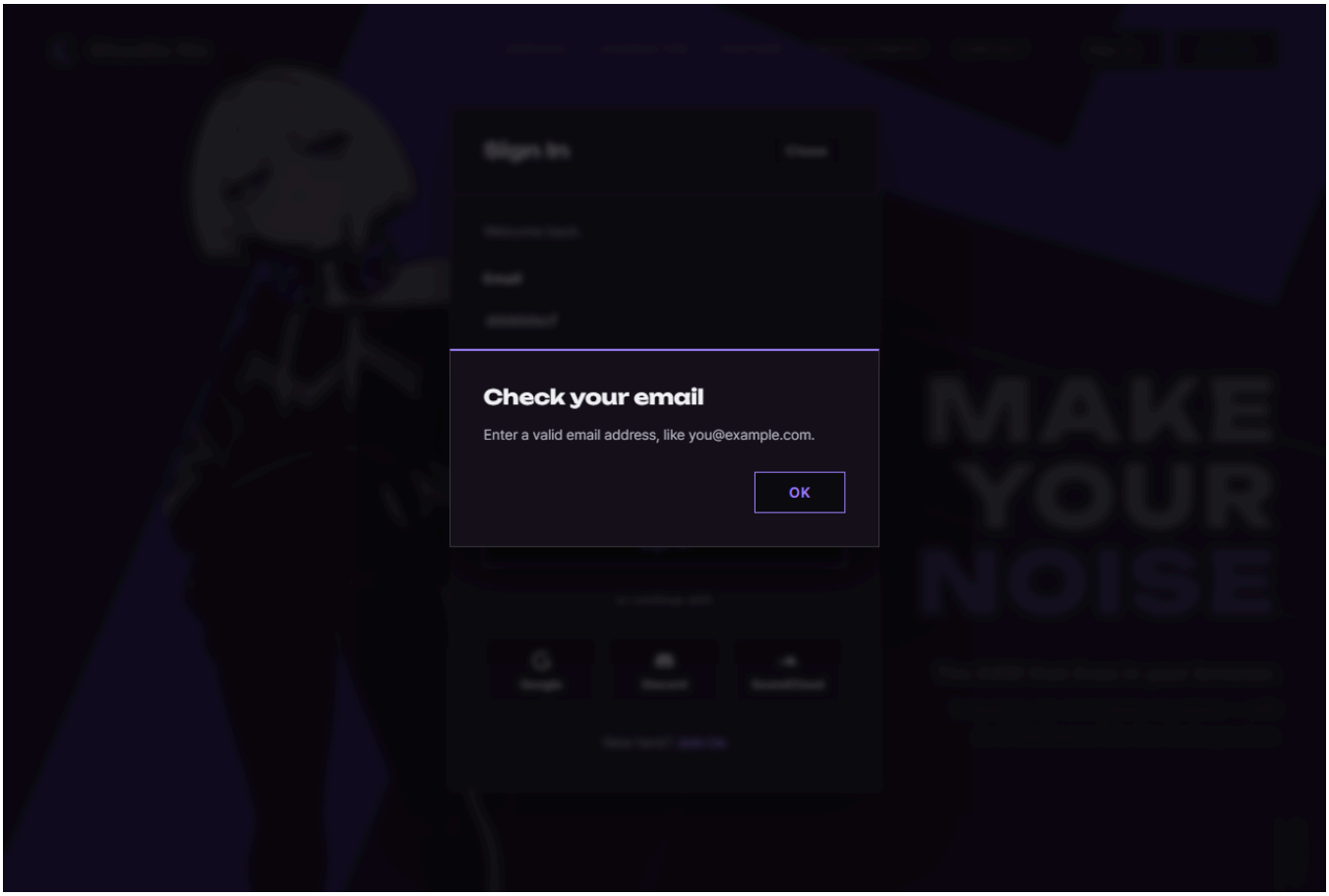
1. ai-notes/javascript/fix - SweetAlert 알림창/18_dialog_showAlert.md
2. ai-notes/javascript/fix - SweetAlert 알림창/19_dialog_initFormAlerts.md
3. ai-notes/javascript/fix - SweetAlert 알림창/20_dialog_confirmDialog.md
4. ai-notes/javascript/fix - SweetAlert 알림창/21_dialog_promptDialog.md

JS 18 — `showAlert()`

항목	내용
파일	`studionx/frontend/static/js/core/dialog.js` (새 파일)
작업	수정 — 브라우저 기본 창 → SweetAlert (사용자 요청 `frontend-reference/images/main page reference/sweetalert수정.pdf`, 2026-10-07)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	17 `initJoinForm()` · 19 `initFormAlerts()`

1. 하는 일

제목(큰 글씨) · 설명 · OK 버튼으로 된 알림창을 띄운다. 모양은 home.html 분위기에 맞췄다 — 먹색 면, 제목은 "Make your noise" · "Sign In" 과 같은 **Unbounded** 글꼴, 모서리 0, 위쪽 2px 보라 선.



부르는 코드	화면
`showAlert('Check your email', 'Enter a valid email address.')`	위 캡처 (위쪽 선 보라)
`showAlert('Oops', 'Something went wrong.', { tone: 'error' })`	위쪽 선만 빨강
`showAlert('Open on a computer', '…', { note: 'Sign in on a computer …', confirmText: 'Got it' })`	설명 아래 작은 글씨 한 줄 + 버튼 글자 "Got it" (목업 09 · 16)
휴대폰 (600px 이하)	아래에서 올라오는 시트 · 버튼 폭 가득 (목업 09 · 13) — `dialog.css`

부르는 코드	화면
로그인 모달이 열려 있을 때	모달 **위에** 뜨고, 닫으면 모달은 그대로 · 커서는 원래 칸으로

2. 언제 쓰나

사용자가 올린 수정 사항: 로그인 모달에서 Email 을 비우거나 형식이 틀리면 **브라우저 기본 말풍선**("이 입력란을 작성하세요.")이 뜬다 → SweetAlert 로, 색감 · 글꼴은 home.html 에 맞게, Prompt · Confirm 창도 같은 형식으로.

함수	하는 일	상태
18 `showAlert`	알림창 (OK 하나) — 작은 화면 안내(목업 09)에도	**이번**
19 `initFormAlerts`	폼의 기본 말풍선을 막고 18 로 보여 줌 — 로그인 모달 · 문의 · 새 프로젝트 폼	다음
20 `confirmDialog`	확인창 (확인 · 취소) — 프로젝트 삭제 (목업 05)	그다음
21 `promptDialog`	입력창 — 회원 탈퇴 비밀번호 (목업 08). 이름 변경은 만들지 않음 (Edit details 모달 그대로, 사용자 결정)	그다음

목업: `docs/page_mockups/sweetalert/` (01 ~ 16, 사용자 검토 2026-10-07)

함께 넣은 것 (코드 외)

파일	내용
`static/vendor/sweetalert2/`	SweetAlert2 **v11.26.25** (MIT, `LICENSE` 함께). CDN 대신 파일로 넣음 — 버전이 고정되고 인터넷이 없어도 됨
`static/css/dialog.css`	모양 (home.html 의 `.modal` · `.btn` · 토큰 그대로)
home · join · myPage · myProject · userInfo `.html`	`<link>` 두 줄 (라이브러리 CSS → `dialog.css` 순서)

3. 코드

import (파일 맨 위)

```
import Swal from '../../vendor/sweetalert2/sweetalert2.esm.min.js';
```

함수

```
export async function showAlert(title, text, options = {}) {
  const tone = options.tone === 'error' ? 'error' : 'info';
  const openModal = document.querySelector('dialog[open]');
  let note;
  if (options.note) {
    note = document.createElement('span');
    note.textContent = options.note; // 글자 그대로 (footer 에 글자를 바로 주면 HTML 로 해석됨)
  }
  await Swal.fire({
    target: openModal || document.body,
    title: title,
    text: text || '',
```

```
    footer: note,
    confirmButtonText: options.confirmText || 'OK',
    buttonsStyling: false,          // 버튼 모양은 우리 .btn
    heightAuto: false,              // <html> · <body> 높이를 건드리지 않음
    scrollbarPadding: false,
    returnFocus: true,
    showClass: { popup: 'snx-dialog-in' },
    hideClass: { popup: 'snx-dialog-out' },
    customClass: {
      popup: 'snx-dialog snx-dialog--' + tone,
      title: 'snx-dialog__title',
      htmlContainer: 'snx-dialog__text',
      actions: 'snx-dialog__actions',
      footer: 'snx-dialog__note',
      confirmButton: 'btn btn--primary',
    },
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
export async function showAlert(title, text = '', { tone = 'info', note, confirmText = 'OK' } = {}) {
  await Swal.fire({
    target: document.querySelector('dialog[open]') ?? document.body,
    title, text, confirmButtonText: confirmText,
    footer: note && Object.assign(document.createElement('span'), { textContent: note }),
    customClass: { popup: `snx-dialog snx-dialog--${tone === 'error' ? 'error' : 'info'}`, … },
    …
  });
}
```

최신 문법	실제 코드	뜻
`text = ''` (매개변수 기본값)	`text: text || ''`	안 넘기면 빈 글자
`{ tone = 'info' } = {}` (구조 분해 + 기본값)	`options = {}` + `options.tone === 'error' ? …`	옵션에서 `tone` 꺼내기
`… ?? document.body`	`openModal || document.body`	없으면 body
`{ title, text }` (속성 줄여 쓰기)	`title: title`	이름이 같으면 한 번만
`` `snx-dialog--${tone}` ``	`'snx-dialog--' + tone`	글자 잇기

문법 풀이

import 문법	뜻
`import Swal from '…'`	그 파일의 **기본 내보내기**(`export default`)를 `Swal` 이라는 이름으로 받음 — `{ }` 없음
`'../../vendor/…'`	`static/js/core/` 에서 두 폴더 위(`static/`) → `vendor/sweetalert2/`

식	뜻
`Swal.fire({ … })`	창을 띄우고, **닫힐 때 끝나는 Promise** 를 돌려줌 → `await` 로 닫힐 때까지 기다림
`document.querySelector('dialog[open]')`	지금 열려 있는 `<dialog>` (없으면 `null`)
`target`	창을 넣을 곳 (SweetAlert2 옵션)

식	뜻
`text`	설명을 **글자 그대로** 넣음 (`html` 옵션과 달리 태그를 해석하지 않음)
`footer: note`	설명 아래 줄. **글자를 그대로 주면 HTML 로 해석**되므로 `<span>` 을 만들고 `textContent` 로 넣어 요소를 넘김. `note` 가 `undefined` 면 줄이 안 생김
`customClass`	SweetAlert2 의 각 부분에 우리 클래스를 붙임 → `dialog.css` 가 모양을 정함
`buttonsStyling: false`	라이브러리의 버튼 색을 끔 → `.btn btn--primary` 모양 그대로

4. 흐름

1. `tone` 을 `'error'` 아니면 `'info'` 로 정함.
2. 열린 `<dialog>` 모달이 있는지 봄.
3. 있으면 그 안에, 없으면 `<body>` 에 창을 띄움 (모양은 `dialog.css`).
4. OK · Enter · Esc · 바깥 클릭으로 닫히면 끝 → 커서는 열기 전 자리로.

5. 왜 이렇게

선택	이유
열린 모달 **안에** 띄움 (`target`)	`showModal()` 로 연 모달은 브라우저 맨 위층에 있고, 모달 밖은 눌리지 않는다. body 에 띄우면 알림창이 **모달 뒤에 깔리고 OK 도 못 누른다** — 로그인 모달이 바로 그 경우
진짜 Esc 가 알림창만 닫음	시험으로 확인: 모달 안에 띄워도 Esc 는 알림창만 닫고 로그인 모달은 남는다
라이브러리 파일을 저장소에 넣음 (CDN 아님)	버전 고정 · 오프라인 개발. 받은 파일에 특정 지역 · 도메인에서 다르게 동작하는 코드가 없는지 확인했다
`.btn btn--primary` · 토큰 재사용	새 색을 만들지 않는다. 라이트 모드도 토큰이 알아서 바뀐다
SweetAlert 기본 아이콘 · 튀는 확대 효과 끔	수정 사항의 형식(제목 · 설명 · 확인)대로. home.html 의 모달처럼 살짝 올라오며 나타남 (움직임 줄이기 설정이면 없음)
`text` 만 받음 (`html` 아님)	서버 오류 문구 등을 넣어도 HTML 로 해석되지 않아 안전
`heightAuto: false`	SweetAlert2 기본은 `<html>` · `<body>` 높이를 바꿔 홈 첫 화면 배치가 흔들릴 수 있다
버튼 글자 기본 `OK`	사이트 글자는 영어 (시켠서만 다국어). 작은 화면 안내는 원래 모달처럼 "Got it"
`note` 를 버튼 **위**에 (`dialog.css` 의 `order`)	SweetAlert 는 footer 를 버튼 아래에 둔다. 보충 설명을 읽은 뒤 버튼을 누르게
휴대폰에서는 아래 시트 (`dialog.css` 600px 이하)	작은 화면 안내(목업 09)는 휴대폰 · 태블릿에서만 뜬다 — 엄지가 닿는 아래쪽, 48px 버튼. 다른 창도 휴대폰에서는 같은 모양

6. 시험 (실제 home.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 11개)

시험	확인한 것
기본	창 · 제목 · 설명 · `OK`, 버튼 클래스 `btn btn--primary`, 커서가 OK 에
모양	제목 글꼴 Unbounded, 모서리 0px, 위쪽 선 `rgb(163, 131, 255)` (`--c-violet-ink`)
닫기	OK → 창 사라짐 · Promise 끝 · 커서가 원래 자리로
`tone: 'error'`	위쪽 선 `rgb(255, 138, 138)` (`--c-error`)

시험	확인한 것
`<b>…</b> <img onerror=…>` 설명	글자 그대로 보이고 태그가 생기지 않음
로그인 모달이 열린 상태	창이 모달 안에 생기고, OK 자리에 실제로 OK 가 보임(가려지지 않음)
진짜 Esc 키	알림창만 닫히고 로그인 모달은 열린 채, 커서는 Email 칸
진짜 Enter 키	닫힘, 모달 그대로
`note` · `confirmText` (목업 반영 뒤 추가)	작은 글씨 줄이 보이고 `<b>` 가 글자 그대로, 줄이 버튼보다 위, 버튼 글자 "Got it", `note` 없으면 줄 없음

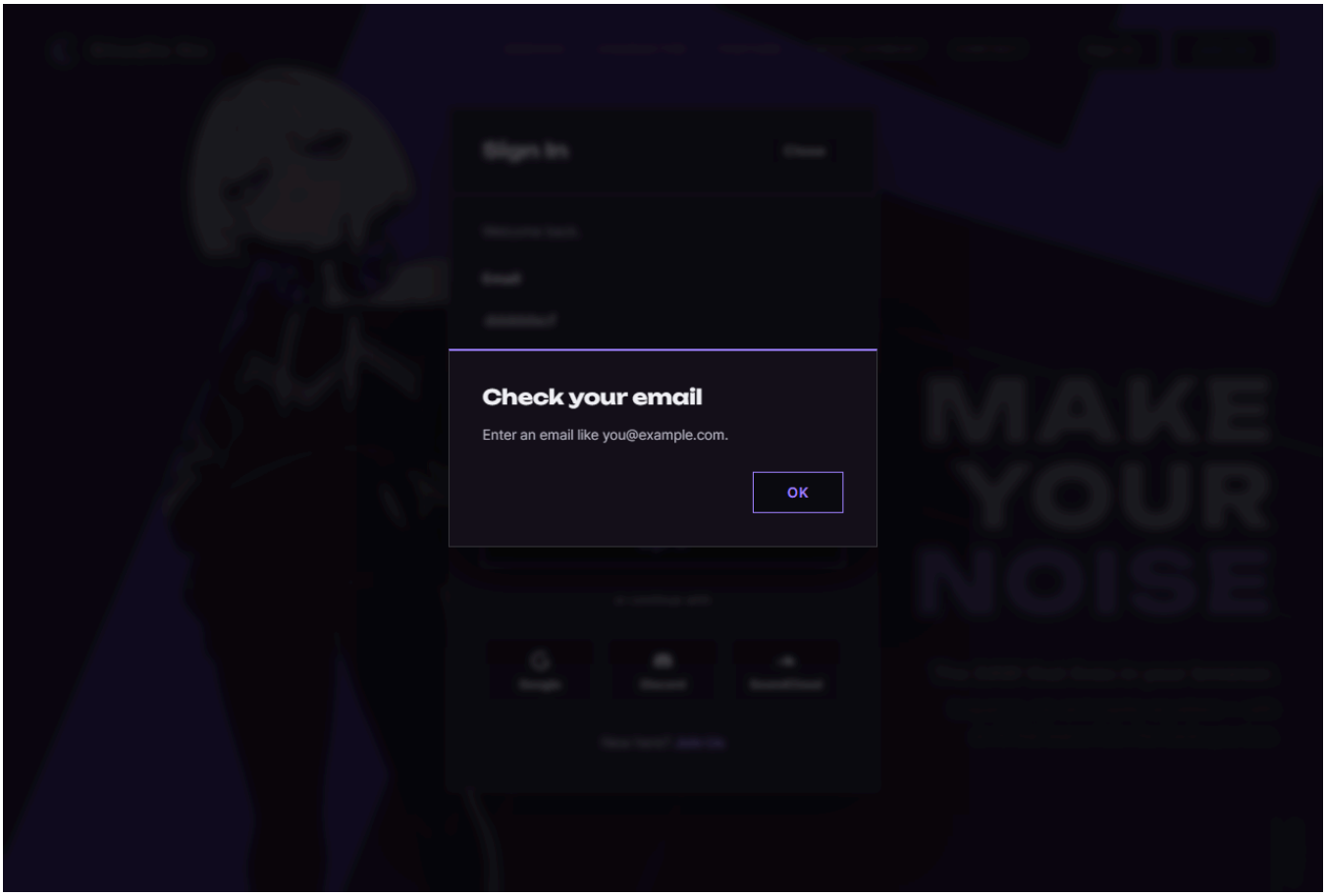
처음 11/11 → `note` 를 더한 뒤 14/14 통과. (Esc · Enter 는 Chrome 개발 도구 기능으로 실제 키를 눌렀다 — 브라우저 기본 동작까지 확인하려고)

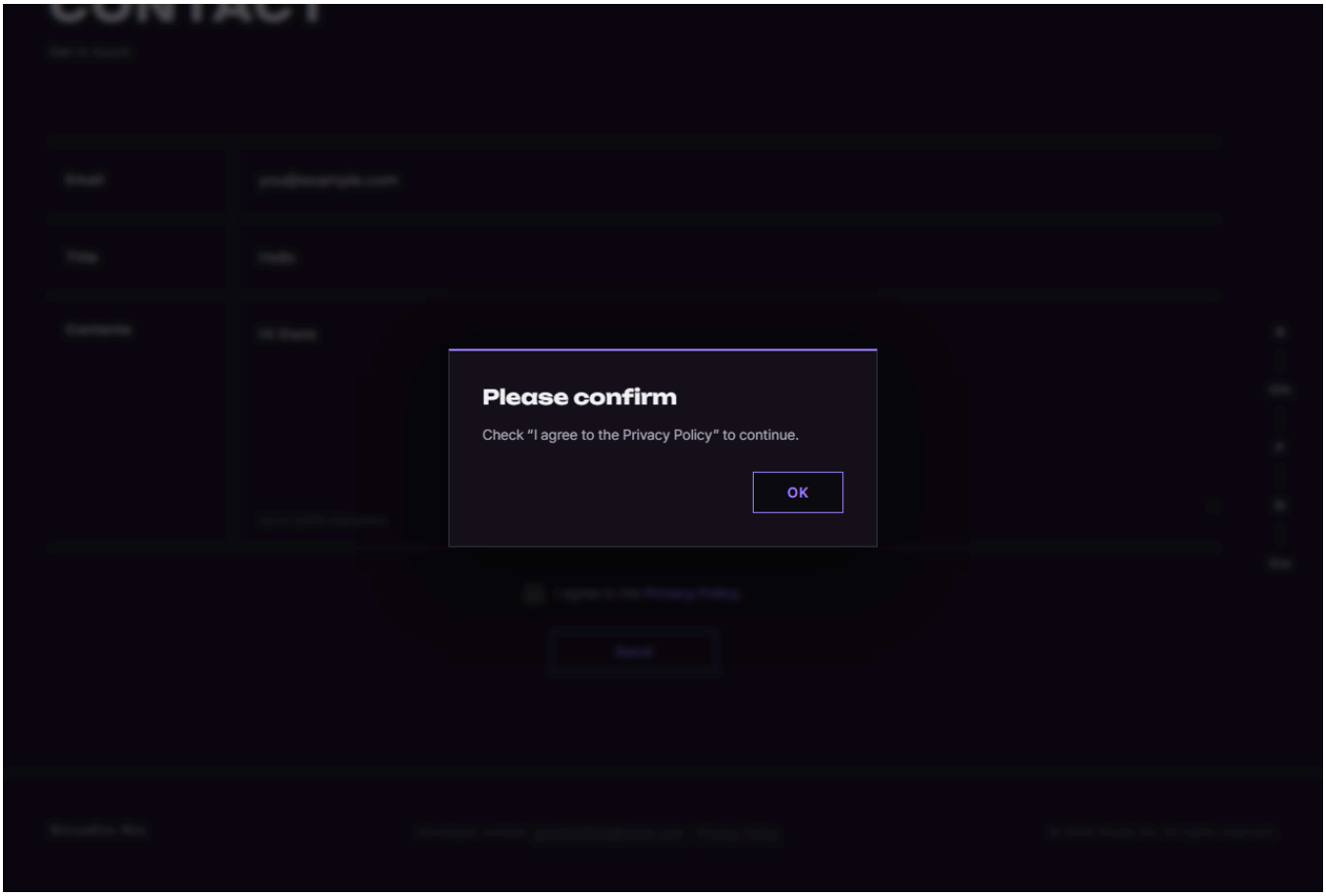
JS 19 — `initFormAlerts()` (+ `alertMessageFor()`)

항목	내용
파일	`studionx/frontend/static/js/core/dialog.js` · 부르는 곳 `pages/home.js` · `join.js` · `myproject.js`
작업	수정 — 브라우저 기본 창 → SweetAlert (사용자 요청 `sweetalert수정.pdf`, 2026-10-07)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	18 `showAlert()` · 20 `confirmDialog()`

1. 하는 일

폼을 보낼 때 빈 칸 · 틀린 형식이 있으면 브라우저가 띄우던 **기본 말풍선**("이 입력란을 작성하세요.", "이메일 주소에 '@'를 포함해 주세요.")을 막고, 18 `showAlert` 알림창으로 보여 준다. OK 를 누르면 그 칸에 커서가 간다.





상황 (사용자 수정 사항 캡처)	전	이제
로그인 모달 Email 빈 칸	말풍선 "이 입력란을 작성하세요."	**Email is required** · Fill in this field to continue.
Email `dddddscf`	말풍선 "이메일 주소에 '@'를 포함해 주세요..."	**Check your email** · Enter an email like you@example.com.
문의 폼 동의 안 함	말풍선	**Please confirm** · Check “I agree to the Privacy Policy” to continue.
새 프로젝트 BPM `5`	말풍선 "값은 20 이상이어야 합니다."	**Check BPM** · Enter a number from 20 to 999.

말풍선은 브라우저 언어(한국어)로 나와 영어 사이트와 섞였고, 모양도 사이트와 달랐다.

2. 언제 쓰나

각 페이지 JS 가 공통 조각(헤더 · 로그인 모달)을 끼운 **뒤** 한 번 부른다.

페이지	대상 폼	대상 아님
home.html	로그인 모달 `#login-form`, 문의 `#contact-form`	
join.html	로그인 모달	가입 폼 `#join-form` — `novalidate` 라 칸 아래 문구(15 ~ 17) 그대로
myProject.html	새 프로젝트 · 정보 수정 · 검색 폼	
userInfo.html	(없음 — 폼이 모두 `novalidate`, P1-10 에서 칸 아래 문구)	

3. 코드

import (`dialog.js` — 18 과 같음, 추가 없음). 부르는 페이지 3곳에는 한 줄씩 추가:

```
import { initFormAlerts } from '../core/dialog.js';
```

함수

```
export function initFormAlerts(root = document) {
  for (const form of root.querySelectorAll('form:not([novalidate])')) {
    if (form.dataset.formAlerts === 'on') continue; // 두 번 연결하지 않음
    form.dataset.formAlerts = 'on';
    let showing = false;

    // 'invalid' 는 위로 퍼지지 않음 → 폼에서 잡으려면 capture(세 번째 인자 true)
    form.addEventListener('invalid', (event) => {
      event.preventDefault(); // 브라우저 말풍선 막기
      if (showing) return; // 한 번 보낼 때 틀린 칸마다 invalid 가 옴 → 첫 칸만 알림
      showing = true;
      const input = event.target;
      const message = alertMessageFor(input);
      input.focus(); // 먼저 커서를 옮김 → 알림창이 닫히면 이 칸으로 돌아옴 (showAlert 의 returnFocus)
      showAlert(message.title, message.text).then(() => {
        showing = false;
      });
    }, true);
  }
}
```

안쪽 도우미 — 틀린 이유 → 문구 (내보내지 않음)

```
function alertMessageFor(input) {
  const label = input.labels && input.labels[0] ? input.labels[0].textContent.replace('*', '').replace(/
\s+/g, ' ').trim().replace(/\.$/, '') : 'This field';
  const v = input.validity;
  if (v.valueMissing && input.type === 'checkbox') {
    return { title: 'Please confirm', text: `Check “${label}” to continue.` };
  }
  if (v.valueMissing) {
    return { title: `${label} is required`, text: 'Fill in this field to continue.' };
  }
  if (v.typeMismatch && input.type === 'email') {
    return { title: 'Check your email', text: 'Enter an email like you@example.com.' };
  }
  if (v.tooShort) {
    return { title: `Check ${label}`, text: `Use at least ${input.minLength} characters.` };
  }
  if (v.tooLong) {
    return { title: `Check ${label}`, text: `Use up to ${input.maxLength} characters.` };
  }
  if (v.rangeUnderflow || v.rangeOverflow || v.stepMismatch || v.badInput) {
    const range = input.min !== '' && input.max !== '' ? `Enter a number from ${input.min} to ${input.ma
x}.` : 'Enter a valid number.';
    return { title: `Check ${label}`, text: range };
  }
```

```
  return { title: `Check ${label}`, text: 'Check this field and try again.' };
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const label = input.labels?.[0]?.textContent.replace('*', '').replace(/\s+/g, ' ').trim().replace(/\.$/, '') ?? 'This field';
…
form.addEventListener('invalid', handler, { capture: true });
…
await showAlert(message.title, message.text);
showing = false;
```

최신 문법	실제 코드	뜻
`input.labels?.[0]?.textContent`	`input.labels && input.labels[0] ? input.labels[0].textContent : …`	이름표가 있을 때만
`… ?? 'This field'`	삼항 연산자의 `: 'This field'`	없으면 기본 글자
`{ capture: true }`	세 번째 인자 `true`	같은 뜻 — 객체로 쓰면 이름이 보임
`async` 처리기 + `await showAlert(…)`	`showAlert(…).then(() => { … })`	닫힌 뒤에 할 일

문법 풀이

import 문법	뜻
`import { initFormAlerts } from '../core/dialog.js'`	`pages/` 에서 한 폴더 위 → `core/dialog.js` 의 이름 있는 내보내기

식	뜻
`'form:not([novalidate])'`	`novalidate` 속성이 **없는** 폼만
`'invalid'` 이벤트	보내기를 누르면 브라우저가 칸마다 검사하고, 틀린 칸**마다** 보냄 (화면 순서대로). 하나라도 있으면 `submit` 은 일어나지 않음
`addEventListener(…, true)`	**capture** — 이벤트가 칸에 닿기 전에 바깥(폼)에서 먼저 받음. `invalid` 는 위로 퍼지지(bubble) 않아서 이렇게 해야 폼 하나에 한 번만 걸면 됨
`event.preventDefault()`	기본 동작(말풍선)을 막음. 보내기는 여전히 막힘
`input.validity`	왜 틀렸는지 — `valueMissing`(빈 칸) · `typeMismatch`(이메일 형식) · `tooShort` · `rangeUnderflow`(min 미만) …
`input.labels[0]`	그 칸의 `<label for=…>`
`.replace(/\s+/g, ' ')`	줄바꿈 · 여러 칸 공백을 한 칸으로 (문의 동의 이름표는 HTML 에서 두 줄)
`.replace(/\.$/, '')`	끝의 마침표 하나 빼기 — 따옴표 안에 넣으므로

4. 흐름

1. 페이지의 `novalidate` 없는 폼마다, 아직 안 붙였으면 `invalid` 받기를 붙인다.
2. 보내기 → 브라우저가 틀린 칸마다 `invalid` 를 보냄.
3. 매번 말풍선을 막는다. 첫 칸이면: 이유로 문구를 정하고 → 그 칸에 커서 → 알림창.
4. 두 번째 칸부터는 무시 (알림은 하나).
5. 알림창이 닫히면 커서가 그 칸으로 돌아오고, 다음 보내기를 받을 준비.

5. 왜 이렇게

선택	이유
`novalidate` 를 붙이지 않고 `invalid` 만 막음	브라우저의 검사 규칙(`required` · `type="email"` · `min/max` · `maxlength`)은 그대로 쓰고 **보여주는 방법만** 바꾼다. HTML 에 규칙이 이미 있으니 JS 에 다시 쓰지 않는다
첫 칸만 알림	알림창이 여러 개 겹치면 안 된다. 고치고 다시 누르면 다음 칸을 알려 준다 (칸 아래 문구 방식인 가입 폼은 17 처럼 한 번에 다 보여 줌)
알림 **전에** 커서를 옮김	처음에는 닫힌 뒤 `input.focus()` 를 불렀는데, SweetAlert 가 그 **뒤에** "열기 전 자리"(보내기 버튼)로 커서를 되돌려 시험 3개가 실패했다. 먼저 옮겨 두면 "열기 전 자리" 가 그 칸이 된다
문구는 이름표에서	칸마다 문구를 HTML 에 따로 적지 않아도 된다. 새 폼이 생겨도 그대로 동작
기본 말풍선 문구(`validationMessage`)를 쓰지 않음	브라우저 언어로 나와서(한국어) 영어 사이트와 섞인다
`data-form-alerts="on"` 표시	두 번 불려도 알림이 두 번 뜨지 않게 (10 비밀번호 토글과 같은 방식)
페이지 3곳에서 부름 (`includePartials` 뒤)	로그인 모달은 공통 조각이라 끼운 뒤에야 폼이 있다

6. 시험 (실제 페이지를 헤드리스 Chrome 으로 열고 그 안에서 실행 · 18개)

페이지	시험	확인한 것
home	로그인 빈 칸	알림 **1개** "Email is required", 말풍선 막힘(`defaultPrevented`), 로그인 요청 0번, 알림은 모달 안
	OK	Email 칸에 커서, 모달 그대로
	`dddddscf`	"Check your email"
	비밀번호 빈 칸 → OK	"Password is required" → Password 칸에 커서
	다 채움	알림 없이 로그인 요청 1번 → 13 의 "Email or password is incorrect." (방해 안 함)
	문의 빈 칸 → OK	"Email is required"(body 에) → 문의 Email 칸에 커서
	문의 동의 안 함 → OK	"Please confirm" · Check “I agree to the Privacy Policy” … → 체크박스에 커서
join	대상	가입 폼에는 안 붙고, 로그인 폼에만 붙음
	가입 폼 빈 칸	알림 없이 칸 아래 "Enter a valid email address." (17 그대로)
	로그인 모달 빈 칸	알림
myProject	새 프로젝트 제목 빈 칸	"Title is required" (새 프로젝트 모달 안)
	BPM `5` → OK	"Check BPM" · Enter a number from 20 to 999. → BPM 칸에 커서

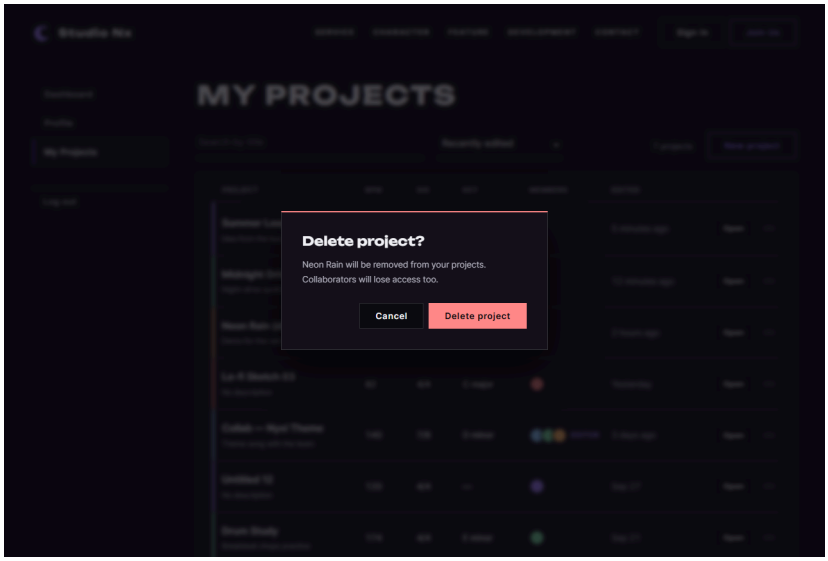
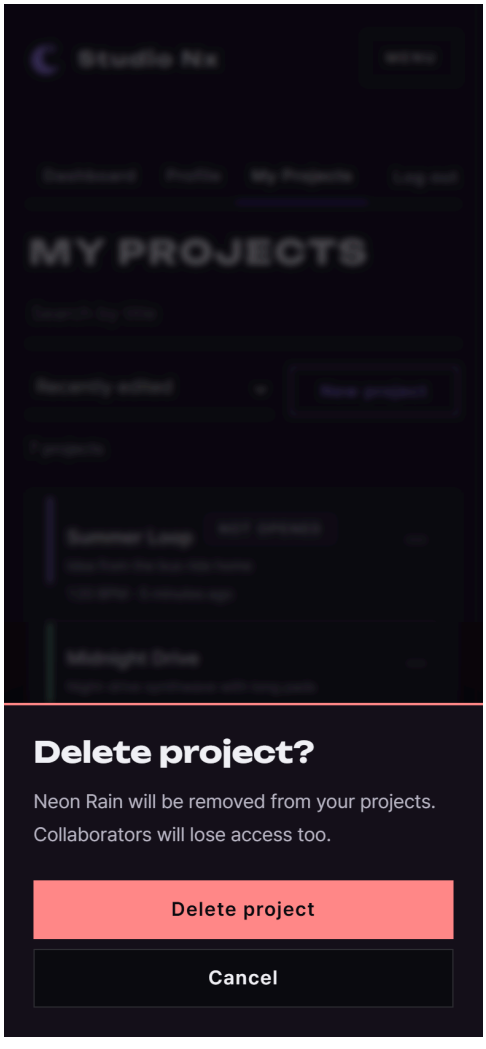
처음 15/18 — 커서 3개 실패(§5), 동의 문구에 줄바꿈이 섞임 → 고친 뒤 **18/18**. 18 `showAlert` 14개, P1-07 시험(13 · 14 · 12 · 16 · 17)도 다시 돌려 모두 통과했다. 시험에서 만든 계정은 지웠다.

JS 20 — `confirmDialog()`

항목	내용
파일	`studionx/frontend/static/js/core/dialog.js`
작업	수정 — 브라우저 기본 창 → SweetAlert (사용자 요청 `sweetalert수정.pdf` "Confirm 창도 SweetAlert 형식", 2026-10-07)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	19 `initFormAlerts()` · 21 `promptDialog()`

1. 하는 일

취소 · 확인 두 버튼짜리 확인창을 띄우고, 사용자가 확인을 눌렀으면 `true`, 아니면 `false` 를 돌려준다. `window.confirm()` 대신 쓴다.

데스크톱	휴대폰 (600px 이하)
	

부르는 코드	화면	돌려주는 값
`await confirmDialog({ title: 'Delete project?', text: '…', confirmText: 'Delete project', danger: true })`	위 캡처. 처음 커서는 **Cancel**	Delete project → `true`

부르는 코드	화면	돌려주는 값
`await confirmDialog({ title: 'Leave this page?', text: '…' })`	보라 OK · Cancel. 처음 커서는 OK	OK → `true`
(어느 쪽이든) Cancel · Esc · 바깥 클릭	닫힘	`false`

2. 언제 쓰나

쓰는 곳	언제 붙이나
내 프로젝트의 **프로젝트 삭제** (목업 05 · 12)	P1-14 `initDeleteProject` 구현 때 — 지금의 `#delete-project-modal` 과 행 메뉴 Delete 의 `command="show-modal"` 을 지우고, Delete 클릭 → `confirmDialog` → `true` 면 삭제 API. `myproject.js` 템플릿 주석에 적어 둠
회원 탈퇴 (목업 08)	비밀번호 칸이 있어서 21 `promptDialog` 로

지금 HTML 모달을 바로 지우지 않은 이유: 삭제 버튼이 **JS 없이** `command` 속성으로 모달을 연다. 모달을 지우면 P1-14 전까지 삭제 버튼이 아무 일도 안 하게 된다.

3. 코드

import (`dialog.js` — 18 과 같음, 추가 없음)

함수

```
export async function confirmDialog(options) {
  const danger = options.danger === true;
  const openModal = document.querySelector('dialog[open]');
  const result = await Swal.fire({
    target: openModal || document.body,
    title: options.title,
    text: options.text || '',
    showCancelButton: true,
    confirmButtonText: options.confirmText || 'OK',
    cancelButtonText: options.cancelText || 'Cancel',
    reverseButtons: true,           // 취소 · 확인 순서 (확인이 오른쪽)
    focusCancel: danger,
    buttonsStyling: false,
    heightAuto: false,
    scrollbarPadding: false,
    returnFocus: true,
    showClass: { popup: 'snx-dialog-in' },
    hideClass: { popup: 'snx-dialog-out' },
    customClass: {
      popup: 'snx-dialog snx-dialog--' + (danger ? 'danger' : 'info'),
      title: 'snx-dialog__title',
      htmlContainer: 'snx-dialog__text',
      actions: 'snx-dialog__actions',
      confirmButton: danger ? 'btn btn--danger' : 'btn btn--primary',
      cancelButton: 'btn btn--ghost',
    },
  });
  return result.isConfirmed === true;
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
export async function confirmDialog({ title, text = '', confirmText = 'OK', cancelText = 'Cancel', dange
r = false }) {
  const { isConfirmed } = await Swal.fire({
    target: document.querySelector('dialog[open]') ?? document.body,
    title, text,
    confirmButtonText: confirmText, cancelButtonText: cancelText,
    customClass: { popup: `snx-dialog snx-dialog--${danger ? 'danger' : 'info'}`, … },
    …
  });
  return isConfirmed;
}
```

최신 문법	실제 코드	뜻
`({ title, text = '', … })` (매개변수에서 구조 분해 + 기본값)	`options` 하나 받고 `options.text || ''`	옵션 꺼내기
`const { isConfirmed } = await …`	`const result = await …` + `result.isConfirmed`	결과에서 칸 하나 꺼내기
`?? document.body`	`openModal || document.body`	없으면 body

문법 풀이

import 문법	뜻
(추가 없음)	18 의 `import Swal from …` 을 같이 씀

식	뜻
`await Swal.fire({ … })`	창이 닫힐 때까지 기다렸다가 **결과 객체**를 받음 — `{ isConfirmed, isDismissed, dismiss }`
`result.isConfirmed`	확인 버튼(또는 확인에 커서가 있을 때 Enter)으로 닫혔으면 `true`. Cancel · Esc · 바깥 클릭은 `false`
`=== true`	혹시 `undefined` 여도 꼭 `true/false` 둘 중 하나로
`showCancelButton: true`	취소 버튼 보이기
`reverseButtons: true`	라이브러리 기본(확인 · 취소)을 뒤집어 취소 · 확인
`focusCancel: danger`	`danger` 가 참이면 처음 커서를 취소 버튼에
`danger ? 'btn btn--danger' : 'btn btn--primary'`	삼항 연산자 — 조건에 따라 둘 중 하나

4. 흐름

1. `danger` 인지 봄, 열린 `<dialog>` 모달이 있는지 봄.
2. 창을 띄움 — danger 면 빨간 선 · 빨간 확인 버튼 · 커서는 취소.
3. 사용자가 닫을 때까지 기다림.
4. 확인으로 닫혔으면 `true`, 아니면 `false` 를 돌려줌. 커서는 열기 전 자리로.

5. 왜 이렇게

선택	이유
`Promise<boolean>` 만 돌려줌	부르는 쪽이 `if (await confirmDialog(…)) { 삭제 }` 한 줄로 쓴다. `window.confirm()` 과 같은 모양이라 바꾸기 쉽다
danger 면 처음 커서가 **취소**	확인창이 뜨자마자 습관적으로 Enter 를 눌러도 지워지지 않는다 (시험으로 확인: Enter → `false`)
danger 아니면 커서가 확인	되돌릴 수 있는 일은 Enter 한 번으로 빠르게
취소 왼쪽 · 확인 오른쪽	기존 HTML 확인창(`.modal__actions`)과 같은 순서. 휴대폰에서는 `dialog.css` 가 위아래로 바꾸고 **확인이 위** (엄지에 가까운 아래가 취소 — 실수로 지우기 어렵게)
Esc · 바깥 클릭 = 취소	사용자가 그냥 닫는 것은 "하지 않겠다" 는 뜻
확인 버튼 글자를 행동으로 (`'Delete project'`)	"OK" 보다 무엇이 일어나는지 분명하다
18 과 설정이 비슷해도 따로 적음	함수 하나만 읽어도 무엇을 하는지 보이게. 세 함수가 모두 끝나면 공통 부분을 묶을지 검토

함께 고친 것 (`dialog.css`): 휴대폰 시트가 화면 **가운데**에 떠 있었다 (목업 09 · 13 포함). SweetAlert 의 화면은 위 · 가운데 · 아래 3줄 격자이고 창이 가운데 줄에 있어서, 아래 줄 · 폭 전체(`grid-row: 3; grid-column: 1 / 4`)로 옮겼다. 목업도 다시 찍었다.

6. 시험 (실제 myProject.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 14개)

시험	확인한 것
danger 모양	제목 · 설명 · "Delete project" · "Cancel", 취소가 왼쪽, 확인 `btn--danger`, 위쪽 선 `rgb(255, 138, 138)`, 처음 커서 Cancel
진짜 Enter (danger)	`false` — 지워지지 않음
Delete project · Cancel 클릭	`true` · `false`
진짜 Esc · 바깥 클릭	`false` · `false`
보통	`btn--primary`, 기본 글자 OK · Cancel, 처음 커서 OK, 보라 선 → 진짜 Enter → `true`
`<img onerror>` 설명	글자 그대로
Edit details 모달이 열린 상태	확인창이 모달 안 · 맨 위에 보이고, Esc → 확인창만 닫힘 (모달 그대로)

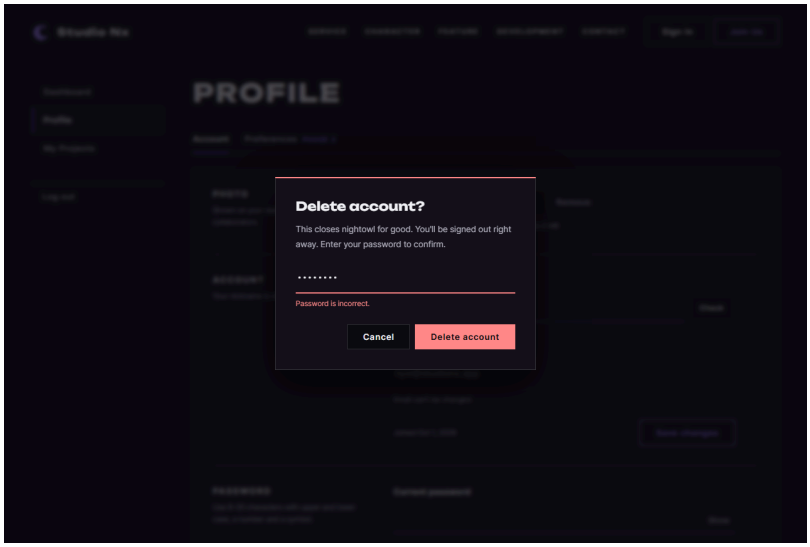
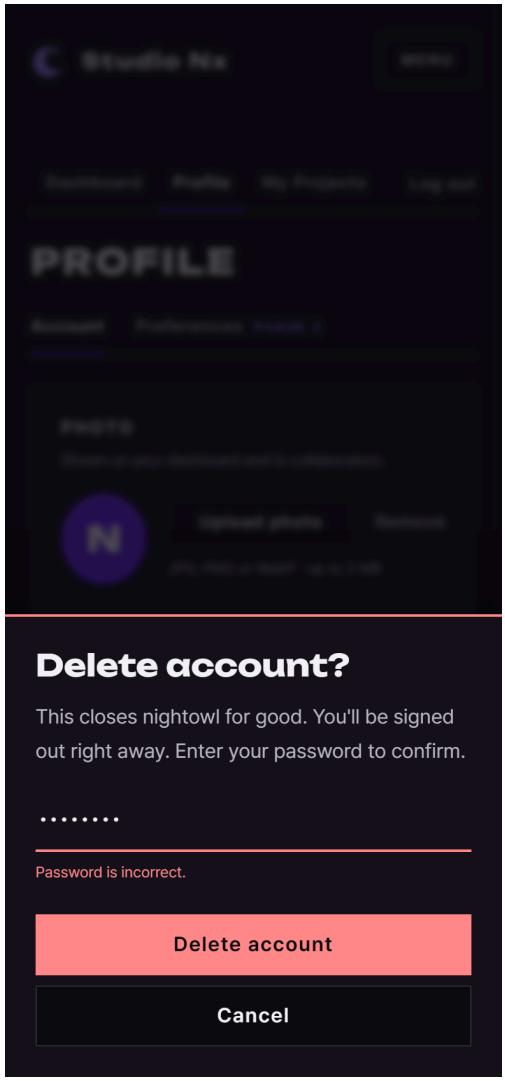
14/14 통과. 처음에는 **시험 코드**의 실수로 멈췄다 — `async` 함수가 Promise 를 돌려주면 `await` 가 그 Promise 가 끝날 때(창이 닫힐 때)까지 기다린다. 객체로 감싸 돌려주게 고쳤다. 18 · 19 시험도 다시 돌려 모두 통과.

JS 21 — `promptDialog()`

항목	내용
파일	`studionx/frontend/static/js/core/dialog.js`
작업	수정 — 브라우저 기본 창 → SweetAlert (사용자 요청 `sweetalert수정.pdf` "Prompt 창도 SweetAlert 형식", 2026-10-07)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	20 `confirmDialog()` · (SweetAlert 수정 마지막)

1. 하는 일

입력칸 하나가 있는 창을 띄우고, 확인을 누르면 **입력한 값**, 취소하면 `null` 을 돌려준다. `window.prompt()` 대신 쓴다. 회원 탈퇴의 비밀번호 확인이 대표적인 쓰임이다 (목업 08 · 14 — 사용자가 고른 "비밀번호만, 체크박스 없음").

데스크톱 (비밀번호 틀림)	휴대폰
	

상황	화면	돌려주는 값
비운 채 Delete account	창 안 빨간 글 "Enter your password." — 닫히지 않음	(기다림)
`check` 가 "Password is incorrect." 를 돌려줌	위 캡처 — 창 안 빨간 글, 바로 다시 입력	(기다림)
`check` 통과	닫힘	입력한 비밀번호 (공백까지 그대로)
글자 칸에 `" Neon Rain "`	닫힘	`"Neon Rain"` (앞뒤 공백 제거)
Cancel · Esc · 바깥 클릭	닫힘	`null`

2. 언제 쓰나

쓰는 곳	언제 붙이나
회원 탈퇴 (userInfo.html, 목업 08 · 14)	P1-10 `initWithdrawForm` 구현 때 — `#withdraw-modal` 과 Delete account 버튼의 `command="show-modal"` 을 지우고 아래처럼. `userinfo.js` 템플릿 주석에 적어 둠
프로젝트 이름 바꾸기	**쓰지 않음** — Edit details 모달 그대로 (사용자 결정 2026-10-07)

탈퇴에서 부를 모양 (P1-10 에서):

```
const password = await promptDialog({
  title: 'Delete account?',
  text: "This closes nightowl for good. You'll be signed out right away. Enter your password to confir
m.",
  inputType: 'password', placeholder: 'Password', confirmText: 'Delete account', danger: true,
  requiredMessage: 'Enter your password.',
  check: async (pw) => {            // 확인을 누를 때 — 문구를 돌려주면 창 안에 보이고 닫히지 않음
    try { await api.delete('/users/me', { body: { password: pw } }); return null; }   // 본문은 body 로
(api.delete 의 두 번째 인자는 옵션)
    catch (err) { return err.status === 401 ? 'Password is incorrect.' : err.message; }
  },
});
if (password !== null) { /* 탈퇴 끝 → 홈으로 */ }
```

3. 코드

import (`dialog.js` — 18 과 같음, 추가 없음)

함수

```
export async function promptDialog(options) {
  const danger = options.danger === true;
  const isPassword = options.inputType === 'password';
  const clean = (value) => (isPassword ? value : value.trim()); // 비밀번호의 공백도 비밀번호의 일부
  const openModal = document.querySelector('dialog[open]');
  const attributes = { 'aria-label': options.placeholder || options.title };
  if (options.maxLength) attributes.maxlength = String(options.maxLength);
  if (isPassword) attributes.autocomplete = 'current-password';

  const result = await Swal.fire({
```

```
    target: openModal || document.body,
    title: options.title,
    text: options.text || '',
    input: isPassword ? 'password' : 'text',
    inputValue: options.value || '',
    inputPlaceholder: options.placeholder || '',
    inputAttributes: attributes,
    inputAutoTrim: false,           // 라이브러리는 기본으로 앞뒤 공백을 지움 → 비밀번호가 바뀌므로 끄고 cl
ean 으로
    inputValidator: (value) => (clean(value) === '' ? (options.requiredMessage || 'Fill in this field.')
: null),
    preConfirm: async (value) => {
      if (!options.check) return clean(value);
      // 확인 중에는 취소 못 함 — 서버가 이미 처리하는데 화면만 "취소" 가 되지 않게 (탈퇴)
      Swal.getCancelButton().disabled = true;
      let message;
      try {
        message = await options.check(clean(value));
      } catch (err) {
        message = err.message || 'Something went wrong. Please try again.';
      }
      Swal.getCancelButton().disabled = false;
      if (message) {
        Swal.showValidationMessage(message); // 창 안 빨간 글, 닫지 않음
        return false;
      }
      return clean(value);
    },
    showLoaderOnConfirm: true,      // check 를 기다리는 동안 버튼 자리에 도는 표시
    allowEscapeKey: () => !Swal.isLoading(),    // 확인 중에는 Esc · 바깥 클릭으로도 닫히지 않음
    allowOutsideClick: () => !Swal.isLoading(),
    showCancelButton: true,
    confirmButtonText: options.confirmText || 'OK',
    cancelButtonText: options.cancelText || 'Cancel',
    reverseButtons: true,
    buttonsStyling: false,
    heightAuto: false,
    scrollbarPadding: false,
    returnFocus: true,
    showClass: { popup: 'snx-dialog-in' },
    hideClass: { popup: 'snx-dialog-out' },
    customClass: {
      popup: 'snx-dialog snx-dialog--' + (danger ? 'danger' : 'info'),
      title: 'snx-dialog__title',
      htmlContainer: 'snx-dialog__text',
      input: 'snx-dialog__input',
      actions: 'snx-dialog__actions',
      loader: 'snx-dialog__loader',
      confirmButton: danger ? 'btn btn--danger' : 'btn btn--primary',
      cancelButton: 'btn btn--ghost',
    },
  });
  return result.isConfirmed ? result.value : null;
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
export async function promptDialog({ title, text = '', inputType = 'text', value = '', placeholder = '', maxLength,
  confirmText = 'OK', cancelText = 'Cancel', danger = false, requiredMessage = 'Fill in this field.', check }) {
  const isPassword = inputType === 'password';
  const clean = (v) => (isPassword ? v : v.trim());
  const { isConfirmed, value: answer } = await Swal.fire({
    target: document.querySelector('dialog[open]') ?? document.body,
    inputAttributes: { 'aria-label': placeholder || title, ...(maxLength && { maxlength: String(maxLength) }), ...(isPassword && { autocomplete: 'current-password' }) },
    preConfirm: async (v) => {
      const message = await check?.(clean(v)).catch((err) => err.message);
      …
    },
    …
  });
  return isConfirmed ? answer : null;
}
```

최신 문법	실제 코드	뜻
매개변수 구조 분해 + 기본값	`options` 하나 + `options.x || 기본값`	옵션 꺼내기
`...(조건 && { 키: 값 })` (조건부 펼치기)	`if (조건) attributes.키 = 값`	조건이 맞을 때만 속성 추가
`check?.(…)`	`if (!options.check) return …`	함수가 있을 때만 부르기
`.catch((err) => err.message)`	`try { … } catch (err) { message = err.message … }`	오류를 문구로
`const { isConfirmed, value: answer } = …`	`result.isConfirmed ? result.value : null`	결과 꺼내기 (`value: answer` = 이름 바꿔 꺼내기)

문법 풀이

import 문법	뜻
(추가 없음)	18 의 `import Swal from …` 을 같이 씀

식	뜻
`const clean = (value) => (isPassword ? value : value.trim())`	안쪽 화살표 함수 — 글자 칸만 앞뒤 공백 제거
`input` · `inputValue` · `inputPlaceholder` · `inputAttributes`	SweetAlert2 입력칸 옵션 — 종류 · 처음 값 · 흐린 안내 글자 · `<input>` 에 붙일 속성
`inputAutoTrim: false`	라이브러리가 값의 앞뒤 공백을 지우는 것을 끔
`inputValidator`	확인을 누르면 먼저 불림. 글자를 돌려주면 그 문구를 보여 주고 닫지 않음, `null` 이면 통과
`preConfirm`	검사를 통과한 뒤 불림. `false` 를 돌려주면 닫지 않음, 값을 돌려주면 그 값으로 닫힘 → `result.value`
`Swal.showValidationMessage(글)`	입력칸 아래 빨간 글
`Swal.getCancelButton().disabled = true`	취소 버튼 잠금
`showLoaderOnConfirm: true`	`preConfirm` 이 끝날 때까지 확인 버튼 자리에 도는 표시

식	뜻
`allowEscapeKey: () => !Swal.isLoading()`	함수로 주면 Esc 를 누를 때마다 물어봄 — 확인 중이면 `false` (닫지 않음)

4. 흐름

1. 종류(글자 · 비밀번호) · danger · 열린 모달을 보고 입력칸 속성을 만든다.
2. 창을 띄움 — 커서는 입력칸.
3. 확인 → 비었으면 창 안 문구 (끝나지 않음).
4. `check` 가 있으면: 취소 · Esc · 바깥 클릭을 잠그고 도는 표시 → `check(값)` → 문구가 오면(또는 오류) 잠금을 풀고 창 안 문구, 계속 입력 / `null` 이면 닫힘.
5. 확인으로 닫혔으면 값, 아니면 `null` 을 돌려준다.

5. 왜 이렇게

선택	이유
`check` 로 **창 안에서** 서버 확인	비밀번호가 틀렸을 때 창이 닫히고 다시 열리면 처음부터 해야 한다. 기존 탈퇴 모달도 틀리면 모달 안에 "Password is incorrect." 를 보였다 (P1-10 템플릿)
`check` 중에는 **취소 · Esc · 바깥 클릭 잠금**	시험에서 찾은 빈틈: 서버가 탈퇴를 처리하는 동안 Cancel 을 누르면 화면은 "취소" 인데 실제로는 탈퇴될 수 있었다
비밀번호는 공백을 지우지 않음 (`inputAutoTrim: false`)	시험에서 찾은 빈틈: 라이브러리가 기본으로 공백을 지워 `" pw "` 가 `"pw"` 로 바뀌었다. **받은 그대로** 서버에 보내 서버가 판단하게 한다
글자 칸은 앞뒤 공백 제거	이름 같은 값은 실수로 넣은 공백이 의미가 없다
`autocomplete="current-password"`	브라우저 · 비밀번호 관리자가 저장된 비밀번호를 채워 줄 수 있다
`aria-label`	화면에 이름표(`<label>`)가 없어서, 화면 읽기 프로그램이 칸 이름을 알 수 있게
처음 커서는 입력칸 (danger 여도)	비밀번호를 쳐야만 확인할 수 있으므로 Enter 실수로 지워질 일이 없다
체크박스 없이 비밀번호만	사용자 결정 (목업 08). 빨간 선 · 빨간 버튼 · 설명 문구 · 비밀번호 입력이 이미 확인 단계 역할

6. 시험 (실제 userInfo.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 18개)

시험	확인한 것
모양	`type="password"` · `autocomplete` · `aria-label` · placeholder "Password", 처음 커서 입력칸, 빨간 선 · 빨간 Delete account · Cancel 왼쪽, 입력칸은 밑줄(상자 아님)
빈 칸	창 안 "Enter your password.", 닫히지 않음
`" pw with spaces "`	그대로 돌려줌
`check` 틀림	기다리는 동안 도는 표시 · Cancel 잠김 · **Esc · 바깥 클릭에도 안 닫힘** → "Password is incorrect." · Cancel 다시 켜짐 · 닫히지 않음
다시 입력 → 통과	값을 돌려줌, `check` 가 받은 값 `Wrong1!x, Right1!x`

시험	확인한 것
`check` 가 오류를 던짐	그 문구("Network down.")를 창 안에
Cancel · 진짜 Esc	`null` · `null`
글자 칸	미리 채움 · `maxlength=100` · 보라 버튼, 진짜 Enter → 앞뒤 공백 뺀 값, 공백만 → "Fill in this field."
`<img onerror>` 설명	글자 그대로

처음 14/16 — §5 의 빈틈 2개(비밀번호 공백, 확인 중 취소)를 고치고 시험 2개를 더해 **18/18**. 18 · 19 · 20 시험도 다시 돌려 모두 통과.